T01 · Anvil 与 GitHub/Git 集成 PRD

版本：1.0；日期：2026-10-10（Asia/Shanghai）；状态：需求初稿，待评审、待认领。

入口：总清单 · 交接记录 · 测试用例 · 公共边界

1. 摘要

将现有 Git 执行能力扩展为可追溯的 GitHub 协作闭环。用户绑定仓库、导入选定 Issue，通过 Anvil 审查固定版本的交付，再以 PR 交付并同步远端结果。纯本地 Git 保留安全合入能力。本任务可以使用 fake backend 独立开发和验收，不等待 Codex adapter。

2. 现状、问题与影响

代码核对点：delivery-evidence-loop-v0@ff12dea4d57639ae1dbd189872d2b27ea56074a7，2026-10-10 复核关键源码，未运行产品测试。

已有 clone/fetch、缓存、worktree、任务分支 push 和 v2 证据。没有 GitHub 连接身份、Issue/PR 映射或同步器。

review.ts 页面 Diff 仍使用浮动分支/merge-base；URL 模式 push 失败仍 markDone；旧合入未接实时 Gate。缓存 ff-only 失败只告警，不能假设缓存 HEAD 等于远端。

这些问题会让用户审错版本、误判完成，或绕过计划中的 PR 流程。

3. 目标与成功标准

目标	验收标准
身份与任务可追溯	从一个 Task 找到稳定仓库身份、来源 Issue、Report/Verification、PR 和人工结论
可信交付	旧 SHA 普通批准、远端失败误 done、GitHub 模式旧路由直推均为 0
可恢复	同一次 PR 写入重复请求/响应丢失不产生重复 PR，unknown 先查询
独立可验收	至少 1 条真实受控 GitHub PR 链路；可用 fake backend，明确不算真实 Codex 样本

4. 用户、范围与非目标

主要用户：仓库所有者审查交付；协作者交付 PR。GitHub 共享工作项与代码，不意味着两人共享 Anvil 数据库和登录。

本期包含仓库绑定、选定 Issue 导入、固定 Diff/实时 Gate、PR、人工动作和同步恢复；包含必要配置与状态 UI。

不做全仓自动导入、评论自动派工、全字段双向同步、自动合并、修改分支保护、Factory、聊天、手机或同时建设 OAuth/App/webhook。完整决策页面重设计与效率对照另记 H01。

5. 功能需求

T01-FR01 仓库身份与连接（P0）

用户希望明确连接的是哪个仓库、以谁的权限操作。

- [ ] Repository 保存 host、稳定 repo ID、full_name、clone URL、目标分支和 local_git/github_pr 模式；本地路径只代表 checkout。
- [ ] 同一仓库 SSH/HTTPS 地址绑定同一身份；重复绑定幂等，重命名/迁移提示确认，不误绑其他仓库。
- [ ] 展示连接身份、权限缺口和状态；可优先使用本机 gh 的受限 API 通路，不读取/导出 token。无权限不写入。

T01-FR02 明确导入 Issue（P0）

用户希望把选定工作项交给本机，而不是全仓自动执行。

- [ ] 导入保存 external_id/number/url、updated_at、目标/验收快照；重复导入唯一映射，不自动派工。
- [ ] 运行中来源更新只提示差异；外部关闭/标签/评论不覆盖执行输入或证明本地验证通过。
- [ ] 一个外部 Issue 可以有多个 Task，每次引用接受的来源版本。

T01-FR03 固定版本 Diff 与 Gate（P0）

用户希望看到和批准的版本与实际验证一致。

- [ ] v2 页面使用 base_commit..head_commit；缺对象/锚点/仓库身份不一致明确失败，不回退浮动 Diff。
- [ ] 裁决及实际合入前核对 Task、分支、HEAD、Report、Verification、clean、目标身份；HEAD/dirty/目标错配阻止批准。
- [ ] Verification 的 failed/partial/not_configured 阻止普通批准；质量类阻塞仅能通过填写理由、二次确认、留审计的明确覆盖裁决处理。缺 Report、dirty、SHA/仓库/目标错配等完整性阻塞不可覆盖。
- [ ] 目标前进产生 base_advanced 完整性阻塞，保留历史验证事实；显式新执行重新准备。
- [ ] 纯本地合入也核对实际目标工作区；Git 临界操作互斥，不能在 Agent 执行或人工等待期间持全仓锁。

T01-FR04 裁决与 PR 交付（P0）

用户希望保留自己的决定，并知道 Git 操作是否真正成功。

- [ ] Decision 与 IntegrationAttempt 分开，绑定 Task、证据版本与操作者；操作重试不制造第二次批准。
- [ ] 每 Task 至多一条权威 Decision；同请求幂等返回原结果，不同后续裁决返回冲突。操作恢复记录新 Attempt，返工创建新 Task，不能改写原裁决。
- [ ] github_pr 使用任务分支 → PR → 人工审查 → 远端确认；服务端拒绝旧 /merge 直推，包括历史任务入口，不自动用管理员 bypass。
- [ ] 首版可跳转 GitHub 手动合入；平台合入按钮在权限、required checks、实时 SHA 核对齐全前不可用。
- [ ] 纯本地 Git 明确“本地交付”。旧 v1 不伪造 v2 证据；历史兼容路径标明限制。

T01-FR05 远端事实与幂等恢复（P0）

用户希望网络失败后继续交付，不丢现场或重复操作。

- [ ] push/PR 写入失败不 done、不删除现场；unknown 先按关联对象查询再重试。

- [ ] 新 PR head 使旧批准失效，关闭未合入不是完成；只确认所选权威 PR/目标后收尾。
- [ ] squash/rebase 不要求 merge commit=任务 head，应核对最后批准 head、merged、merge commit 与目标可达事实。
- [ ] 一开放 PR 可承接返工，各 Task/head/审查独立保留；外部自动关闭 Issue 不反推本地通过。

T01-FR06 同步与最小操作界面（P0）

用户希望知道数据多久没更新、下一步能做什么。

- [ ] 首版出站轮询和手动刷新，显示最后同步时间、限流/断网/过期；重复同步不产生写副作用。
- [ ] 展示来源 Issue、版本、PR、实测和阻塞；GitHub checks 与 Runner 检查分开。
- [ ] 保留计时数据；打开 PR 只可保存计时草稿，明确提交真实裁决后才是有效判断，不以收满旧 UI 样本阻止基础功能完成。

以上均为本期闭环必需项。Webhook、批量操作和完整新决策布局是后续增强，不通过优先级降级来豁免正确性。

6. 非功能要求

幂等状态持久化、错误脱敏、已验证 SHA 使用完整值；默认单机部署。连接凭据不注入 Agent，不把源代码/原始日志默认写 GitHub。

同步有退避、单仓请求限额和人工刷新保护，具体周期在设计中冻结；断网 UI 保留旧事实并标旧。

迁移有备份/回滚说明；Git common-dir 身份和跨进程互斥可测试，拒绝并发污染。

7. 架构与实现边界

保留 Core/Fastify-SQLite/Runner/React 四包。Server 管 GitHub connector、Git Gate、审查、同步和操作记录；Runner 管 worktree/锚点/独立验证；Web 展示来源和交付。

T01 协调 worktree.ts、repocache 与 Git 操作；T02 只运行 CLI。Core/executor/迁移先指定整合人，契约补丁先于两边实现。T01 不解析 Codex 私有事件。

8. API 建议（未实现）

操作	拟议接口	成功结果
绑定仓库	POST /api/repositories	201 Repository；重复返回已绑定对象
导入 Issue	POST /api/repositories/:id/import-issue	200/201 本地映射，不派工
固定 Diff	GET /api/tasks/:id/diff（扩展既有）	200 base/head/diff/truncated
PR 交付	POST /api/tasks/:id/pull-request	202 operation_id，查询最终结果
提交裁决	POST /api/tasks/:id/decisions	200 Decision，非已合入

请求示例：{"issue_number":12,"source_updated_at":"2026-10-10T00:00:00Z"}。响应示例：{"issue_id":"local-id","source_snapshot_id":"snapshot-id","disposition":"imported"}。

写请求携带 Idempotency-Key、expected_head/expected_version；400 输入非法，403 权限，409 版本/状态/载荷冲突，503 暂不可用；错误有 code/message/retryable。

本期人类入口仅本机，仍预留身份校验；T05 后所有远程 HTTP/WS 统一人类认证。daemon/task 权限不混用。

9. 数据建议（未实现）

实体	关键字段 / 类型	约束
Repository	id/host/external_id/full_name/clone_url/target_branch:string；mode:enum	workspace 1:N；host+external_id 唯一
ExternalIssueLink / SourceSnapshot	repo_id/external_id/local_issue_id/hash:string；number:int；updated_at:timestamp	repo+external_issue 唯一，执行固定快照
PullRequestLink	task_id/repo_id/external_id/head/base/merge_commit?:string；number:int；state:enum	多 Task 可关联开放 PR，保留版本历史
Decision / Attempt	task/head/report/run/actor 引用；result:enum；time:timestamp	Decision.task_id 唯一；Attempt 可多次，裁决与操作分离
ExternalOperation	key/actor/action/target:string；expected_version:string?；state:enum	key 唯一，unknown 先核对

Task completed、Report delivered、Verification passed、人工 approved、PR merged 是独立事实。

10. 交互与状态

连接仓库 → 导入 Issue → 派发既有执行器 → 查看固定证据 → 建/关联 PR → 明确裁决 → 查看远端状态。

空连接、权限不足、缺对象、base_advanced、验证失败、PR 写入未知、断网、关闭未合入均有原因和安全下一步。

最小改动沿用现有面板，不把本任务变成整个 UI 重做。

11. 集成与依据

GitHub API 连接方案在实现时核对官方权限与版本；首版建议 gh 适配。外部评论/标签属于资料，不是平台指令。

源码依据：review.ts、worktree.ts、repocache.ts。

T02 不构成本任务独立开发前置；双方共同使用 COMMON。

12. 异常与恢复

情形	期望
base 前进、HEAD 移动、dirty、目标错	阻止旧批准，保留事实，显式重新准备
push 拒绝、PR 超时	不 done，保留现场，按 operation 查询再重试
权限不足、保护拒绝	不修改保护或换身份自动突破
限流/断网	退避、标旧，恢复后收敛
仓库解绑、对象缺失	保留历史，拒绝不确定操作
外部文本要求执行	不派活、不扩大权限

13. 测试与完成定义

执行 T01 用例，包含真实 Git、接口替身和受控 GitHub PR；fake backend 不计 Codex 真样本。

全部本期 FR、必要自动化、真实 PR、独立审查和产品验收有证据才完成；合并/发布另记。

T01 与 T02 各自通过后执行 J01，J01 才是 T03 前置。H01 效率对照不阻止基础正确性验收。

14. AI 实施顺序与交接

先冻结仓库/操作/裁决契约，修固定 Diff 和误 done，封住 GitHub 旧直推入口，再实现绑定、导入、PR 和同步 UI，最后异常/真实验收。

不重写调度器，不运行生产仓库写入探针。开发前填写 T01 交接，工作段结束记录完成 FR、未完成、测试、审查和下一步。

待设计确认：gh 连接边界、绑定迁移方式、目标分支更新策略、跨进程锁范围、最小界面。跨任务字段须双方确认，不能以单分支编译代替联调。

15. 版本记录

版本	日期	变化	决策状态
1.0	2026-10-10	从总 PRD 拆出 Git/GitHub 独立主题；允许与 T02 并行，Factory 后置	初稿待评审；未认领、未实施